

CAPSTONE PROJECT REPORT

SDET TRAINING PROGRAM

PARABANK AUTOMATION TESTING FRAMEWORK

SECTION 1-TEST PLAN

Prepared By

Manish Chaudhary

GitHub Repository

<https://github.com/manish-chaudhary-1409/parabank-capstone-automation-framework>

Project Information

Attribute	Details
Project Name	ParaBank Banking Application Automation Framework
Application Under Test (AUT)	ParaBank
Application URL	https://parabank.parasoft.com/parabank
Testing Type	Functional Automation Testing
Automation Tool	Selenium WebDriver
Framework	TestNG
Programming Language	Java
Design Pattern	Page Object Model (POM)
Build Tool	Maven

Reporting Tool	Extent Reports
Logging Framework	Log4j2
Version Control	Git & GitHub
CI/CD Tool	Jenkins
Containerization Tool	Docker
Prepared By	Manish Chaudhary

1. Introduction

The banking industry requires highly reliable, secure, and scalable software systems capable of handling sensitive customer information and financial transactions. Any defect within a banking application can directly impact customer trust, transaction integrity, and business operations.

The ParaBank Banking Application Automation Framework was developed to automate the validation of critical banking workflows using Selenium WebDriver, Java, TestNG, Maven, Jenkins, Docker, and GitHub.

This Test Plan defines the testing strategy, scope, objectives, environment, deliverables, risks, and execution approach for validating the ParaBank application through automation testing.

The framework follows industry-standard automation practices including Page Object Model (POM), Data Driven Testing, centralized reporting, logging, CI/CD integration, and containerized execution.

2. Project Objectives

Functional Objectives

- Validate customer registration functionality.
- Verify login functionality using valid credentials.
- Validate login failure using invalid credentials.

- Verify account overview functionality.
- Validate transfer funds functionality.
- Verify bill payment processing.
- Validate negative login scenarios.
- Verify empty login validation.

Framework Objectives

- Develop a reusable automation framework.
- Implement Page Object Model (POM).
- Implement Data Driven Testing.
- Generate Extent Reports automatically.
- Enable screenshot capture during execution.
- Integrate Jenkins CI/CD pipeline.
- Support Docker-based execution.
- Maintain source code through GitHub.

3. Scope of Testing

In Scope

Customer Registration

- New Customer Registration
- Registration Validation

Authentication

- Valid Login
- Invalid Login
- Empty Login Validation

Banking Services

- Account Overview Validation
- Transfer Funds Validation

- Bill Payment Validation

Framework Features

- Data Driven Testing
- Screenshot Capture
- Extent Report Generation
- Jenkins Integration
- Docker Integration
- Negative Testing

Out of Scope

The following activities are not included in the current testing phase:

- API Testing
- Database Testing
- Mobile Application Testing
- Performance Testing
- Load Testing
- Stress Testing
- Security Testing
- Penetration Testing

4. Test Strategy

The project follows an automation-first testing strategy using Selenium WebDriver and TestNG.

The following testing methodologies are implemented:

Functional Testing

Validation of business workflows against expected system behavior.

Regression Testing

Verification that application functionality remains stable after changes.

Data Driven Testing

Execution of test scenarios using external data sources.

Negative Testing

Validation of application behavior using invalid and unexpected inputs.

5. Test Environment

Component	Configuration
Operating System	macOS
Browser	Google Chrome
Programming Language	Java 21
IDE	Eclipse
Automation Tool	Selenium WebDriver
Framework	TestNG
Build Tool	Maven
Reporting Tool	Extent Reports
Logging Framework	Log4j2
Version Control	GitHub
CI/CD Tool	Jenkins
Containerization Tool	Docker

6. Test Scenarios

Functional Test Scenarios

Test Case ID	Test Scenario	Priority
TC_REG_001	Register New Customer	High
TC_LOGIN_001	Login with Valid Credentials	High
TC_LOGIN_002	Login with Invalid Credentials	Medium
TC_LOGIN_003	Login with Empty Credentials	Medium
TC_ACC_001	Account Overview Validation	High
TC_TRF_001	Transfer Funds Validation	High
TC_BILL_001	Bill Payment Validation	High

Planned Negative Test Scenarios

Test Case ID	Scenario	Priority
TC_NEG_001	Invalid Username and Password	High
TC_NEG_002	Empty Username and Password	High
TC_NEG_003	Transfer Funds Validation Checks	High

TC_NEG_004	Bill Payment Validation Checks	High
------------	--------------------------------	------

7. Test Data Requirements

Registration Data

- First Name
- Last Name
- Address
- City
- State
- Zip Code
- Phone Number
- SSN
- Username
- Password

Login Data

- Valid Credentials
- Invalid Credentials

Banking Data

- Transfer Amount
- Bill Payment Amount

8. Entry Criteria

Testing activities may begin when:

- ParaBank application is accessible.
- Test environment is configured.
- Selenium framework setup is completed.

- Maven dependencies are installed.
- Test data is available.
- Jenkins environment is configured.
- Docker environment is available.

9. Exit Criteria

Testing activities will be considered complete when:

- All planned test cases are executed.
- Critical functionalities are validated.
- Defects are documented.
- Reports are generated successfully.
- Execution summary is prepared.
- Deliverables are completed.

10. Test Deliverables

Documentation Deliverables

- Test Plan
- Test Analysis Report
- Defect Report
- Final Project Report

Automation Deliverables

- Selenium Automation Scripts
- Page Object Classes
- TestNG Suite Files
- Test Data Files

Reporting Deliverables

- Extent Reports

- TestNG Reports
- Execution Screenshots

DevOps Deliverables

- Jenkinsfile
- Dockerfile
- pom.xml

11. Risk Analysis and Mitigation

Risk	Impact	Mitigation Strategy
Application Downtime	High	Re-execute tests after recovery
Dynamic Test Data	Medium	Use runtime-generated usernames
Duplicate Registration Data	Medium	Generate unique user data
Browser Compatibility Issues	Medium	Browser validation
Driver Version Mismatch	Medium	Use WebDriverManager
Slow Application Response	Medium	Explicit Waits
Internet Connectivity Issues	Medium	Verify connectivity before execution

12. Roles and Responsibilities

Role	Responsibility
Automation Engineer	Framework Development
Tester	Test Execution
Developer	Defect Resolution
Reviewer	Review and Approval

13. Expected Deliverables

Framework Deliverables

- ✓ Selenium Automation Framework
- ✓ Page Object Model Implementation
- ✓ Data Driven Testing
- ✓ Reporting and Logging Integration

DevOps Deliverables

- ✓ Jenkins CI/CD Integration
- ✓ Docker Containerization

Documentation Deliverables

- ✓ Test Plan
- ✓ Test Analysis Report
- ✓ Defect Report
- ✓ Final Project Report

14. Approval

Name	Role
------	------

Manish Chaudhary	SDET Trainee
Vaishali	Trainer / Reviewer

Document Status

 READY FOR PROJECT EXECUTION

SECTION 2 – TEST CASE ANALYSIS REPORT

Project Information

Attribute	Details
Project Name	ParaBank Automation Testing Framework
Application Under Test	ParaBank Banking Application
Framework Type	Hybrid Automation Framework
Automation Tool	Selenium WebDriver
Framework	TestNG
Programming Language	Java
Build Tool	Maven
Reporting Tool	Extent Reports
CI/CD Tool	Jenkins
Containerization	Docker

Repository	GitHub
Prepared By	Manish Chaudhary

1. Introduction

The purpose of this Test Case Analysis Report is to evaluate the execution results, coverage, effectiveness, and overall performance of the ParaBank Automation Testing Framework.

The automation suite was executed against the ParaBank Banking Application to validate critical business workflows. The framework successfully executed functional and negative test scenarios while generating detailed execution reports through Extent Reports.

The project also demonstrates integration with Jenkins for CI/CD automation and Docker for containerized execution.

2. Framework Architecture Analysis

The framework follows a layered architecture to improve maintainability, scalability, and reusability.

Base Layer

Responsible for:

- Browser initialization
- Driver management
- Setup and teardown operations

Class Used

- BaseTest.java

Page Layer

Responsible for maintaining web element locators and page actions.

Page Classes

- RegisterPage.java
- LoginPage.java
- AccountOverviewPage.java
- TransferFundsPage.java
- BillPaymentPage.java

Benefits:

- Reduced code duplication
- Improved maintainability
- Better readability

Utility Layer

Provides reusable helper methods.

Utility Classes

- ConfigReader.java
- CSVUtil.java
- WaitUtil.java
- ScreenshotUtil.java
- ExtentReportManager.java

Benefits:

- Centralized configuration management
- Reusable utility functions
- Easier maintenance

Test Layer

Contains actual automation scenarios.

Test Classes

- RegistrationTest.java
- LoginTest.java
- AccountOverviewTest.java
- TransferFundsTest.java
- BillPaymentTest.java
- NegativeTest.java

3. Test Execution Summary

Execution Result

Metric	Count
Total Test Cases Executed	7
Passed	7
Failed	0
Skipped	0
Pass Percentage	100%

The automation framework achieved a 100% execution success rate during the final execution cycle.

4. Functional Test Case Analysis

TC-01 Registration Validation

Objective

Verify successful customer registration and login.

Execution Steps

1. Launch ParaBank Application

2. Navigate to Registration Page
3. Enter customer information
4. Submit registration form
5. Logout
6. Login using generated credentials

Expected Result

Customer should be registered successfully.

Actual Result

Customer registration completed successfully and login was verified.

Status

PASS

Evidence

Insert Registration Execution Screenshot

TC-02 Login Page Validation**Objective**

Verify login page functionality.

Expected Result

Login page should load successfully.

Actual Result

Login page loaded correctly.

Status

PASS

Evidence

Insert Login Screenshot

TC-03 Account Overview Validation

Objective

Verify account overview page accessibility.

Expected Result

Account information should be displayed.

Actual Result

Account overview displayed successfully.

Status

PASS

Evidence

Insert Account Overview Screenshot

TC-04 Transfer Funds Validation

Objective

Verify transfer funds functionality.

Expected Result

Transfer Funds page should be accessible.

Actual Result

Transfer Funds module validated successfully.

Status

PASS

Evidence

Insert Transfer Funds Screenshot

TC-05 Bill Payment Validation

Objective

Verify bill payment functionality.

Expected Result

Bill payment should complete successfully.

Actual Result

Bill payment workflow executed successfully.

Status

PASS

Evidence

Insert Bill Payment Screenshot

5. Negative Test Case Analysis

TC-06 Invalid Login Validation

Objective

Verify application behavior for invalid credentials.

Expected Result

Application should reject invalid login attempts.

Actual Result

Application handled invalid credentials successfully.

Status

PASS

Evidence

Insert Invalid Login Screenshot

TC-07 Empty Login Validation

Objective

Verify login validation when credentials are empty.

Expected Result

Application should display validation message.

Actual Result

Validation handled correctly.

Status

PASS

Evidence

Insert Empty Login Screenshot

6. Coverage Analysis**Functional Coverage**

Module	Covered
Registration	Yes
Login	Yes
Account Overview	Yes
Transfer Funds	Yes
Bill Payment	Yes
Negative Login	Yes

Coverage Achieved: 100%

7. Reporting Analysis

The framework integrates Extent Reports for execution reporting.

Report Features

- Pass/Fail Status
- Execution Logs
- Test Summary
- Timestamp Tracking

Outcome

Extent Reports generated successfully.

Evidence

Insert Extent Report Screenshot

8. GitHub Repository Analysis

The framework source code is maintained through GitHub.

Repository Features

- Version Control
- Commit History
- Source Code Management
- Collaboration Support

Repository Link

<https://github.com/manish-chaudhary-1409/parabank-capstone-automation-framework>

Evidence

Insert GitHub Repository Screenshot

9. Jenkins CI/CD Analysis

Jenkins was integrated to automate project execution.

Pipeline Activities

- Source Code Checkout
- Build Validation
- Test Execution
- Result Publishing

Benefits

- Automated execution

- Continuous Integration
- Reduced manual effort

Evidence

Insert Jenkins Dashboard Screenshot

Insert Jenkins Pipeline Success Screenshot

Insert Jenkins Stages Screenshot

10. Docker Integration Analysis

Docker was used to host Jenkins and support containerized execution.

Validation Performed

- Docker Image Verification
- Docker Container Verification
- Jenkins Container Availability

Benefits

- Environment Consistency
- Simplified Deployment
- Scalability

Evidence

Insert Docker Images Screenshot

Insert Docker Container Screenshot

11. Key Observations

Strengths

- ✓ Reusable Framework Design
- ✓ Page Object Model Implementation
- ✓ Data Driven Testing Support
- ✓ Extent Report Integration

- ✓ Jenkins Integration
- ✓ Docker Integration
- ✓ GitHub Version Control

Improvement Opportunities

- ✓ Enhanced Validation Coverage
- ✓ Additional Negative Scenarios
- ✓ Database Validation
- ✓ API Integration Testing

12. Conclusion

The ParaBank Automation Framework successfully validated all planned business workflows. The automation suite achieved a 100% pass rate and demonstrated effective implementation of Selenium WebDriver, TestNG, Maven, Jenkins, Docker, GitHub, and reporting mechanisms.

The framework meets project objectives and provides a scalable foundation for future automation enhancements.

SECTION 3 – DEFECT REPORT

Project Information

Attribute	Details
Project Name	ParaBank Automation Testing Framework
Application Under Test	ParaBank Banking Application
Defect Tracking Tool	Jira

Testing Type	Functional & Automation Testing
Prepared By	Manish Chaudhary
Repository	https://github.com/manish-chaudhary-1409/parabank-capstone-automation-framework

1. Introduction

The objective of this Defect Report is to document defects and validation gaps identified during the testing and automation execution of the ParaBank Banking Application.

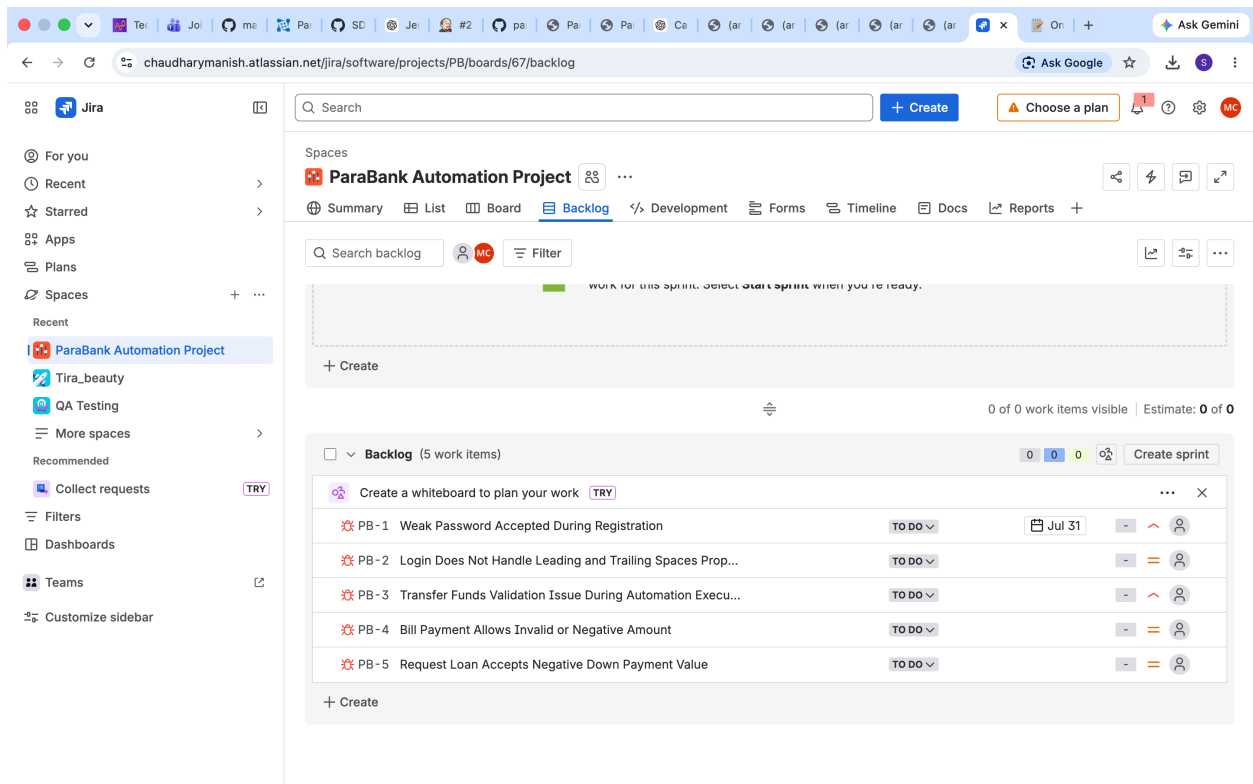
Defects were logged in Jira and analyzed based on severity, priority, business impact, and resolution status.

The identified defects primarily represent validation gaps and business rule improvements that can enhance the reliability and usability of the banking application.

2. Defect Summary

Defect ID	Module	Summary	Severity	Priority	Status
PB-001	Registration	Weak password accepted during registration	Medium	High	Open

PB-002	Login	Leading and trailing spaces not handled properly	Low	Medium	Open
PB-003	Transfer Funds	Validation issue observed during automation execution	High	High	Closed
PB-004	Bill Payment	Invalid or negative amount validation can be improved	Medium	Medium	Open
PB-005	Request Loan	Negative down payment validation can be improved	Medium	Medium	Open



3. Jira Defect Tracking Evidence

Jira Defect Dashboard

Insert Jira Screenshot Here

The Jira dashboard contains all identified defects and their respective statuses.

Tracked Defects:

- PB-001
- PB-002
- PB-003
- PB-004
- PB-005

4. Detailed Defect Analysis

PB-001 – Weak Password Accepted During Registration

Defect Information

Attribute	Value
Defect ID	PB-001
Module	Registration
Severity	Medium
Priority	High
Status	Open
Reported By	Manish Chaudhary

Description

The registration module accepts weak passwords without enforcing sufficient password complexity requirements.

Users can successfully register accounts using passwords that do not meet recommended security standards.

Preconditions

- User is on Registration Page.
- Registration form is available.

Steps to Reproduce

1. Open ParaBank Registration Page.
2. Enter valid customer details.
3. Enter a weak password.
4. Submit registration form.

Expected Result

The application should reject weak passwords and display a validation message.

Actual Result

Registration is completed successfully using a weak password.

Business Impact

Weak password acceptance may increase security risks and expose user accounts to unauthorized access.

PB-002 – Leading and Trailing Spaces Handling Issue

Defect Information

Attribute	Value
Defect ID	PB-002
Module	Login
Severity	Low
Priority	Medium
Status	Open

Description

The login module does not consistently trim leading and trailing spaces from user credentials before authentication.

Steps to Reproduce

1. Open Login Page.
2. Enter valid username with leading spaces.
3. Enter valid password.
4. Click Login.

Expected Result

Spaces should be trimmed automatically and login should proceed normally.

Actual Result

Login validation behaves inconsistently.

Business Impact

May cause unnecessary login failures and poor user experience.

PB-003 – Transfer Funds Validation Issue

Defect Information

Attribute	Value
Defect ID	PB-003
Module	Transfer Funds
Severity	High
Priority	High
Status	Closed / Fixed

Description

During automation execution, validation issues were observed within the Transfer Funds workflow.

Additional verification logic was required to ensure correct transaction confirmation.

Steps to Reproduce

1. Login to application.
2. Navigate to Transfer Funds.
3. Execute fund transfer.
4. Verify transfer confirmation.

Expected Result

Transfer confirmation should be validated correctly.

Actual Result

Automation validation issue observed.

Resolution

Additional synchronization and verification logic was implemented.

Business Impact

Transfer fund workflows represent critical banking transactions and require accurate validation.

PB-004 – Bill Payment Validation Gap

Defect Information

Attribute	Value
Defect ID	PB-004
Module	Bill Payment
Severity	Medium
Priority	Medium
Status	Open

Description

The Bill Payment module can be improved by implementing stronger validation for invalid or negative payment amounts.

Steps to Reproduce

1. Navigate to Bill Payment.
2. Enter negative amount.

3. Submit transaction.

Expected Result

Negative values should be rejected.

Actual Result

Validation controls can be improved.

Business Impact

Improper validation may result in inaccurate financial transactions.

PB-005 – Negative Down Payment Validation Gap

Defect Information

Attribute	Value
Defect ID	PB-005
Module	Request Loan
Severity	Medium
Priority	Medium
Status	Open

Description

The Request Loan functionality should reject negative down payment values.

Steps to Reproduce

1. Navigate to Request Loan module.
2. Enter negative down payment value.
3. Submit request.

Expected Result

System should reject negative values.

Actual Result

Validation can be improved.

Business Impact

Improper loan request validation may impact data accuracy and business logic.

5. Defect Severity Analysis

Severity	Count
High	1
Medium	3
Low	1

6. Defect Status Analysis

Status	Count
Open	4
Closed	1

7. Defect Priority Analysis

Priority	Count
High	2
Medium	3

8. Root Cause Analysis

The majority of identified defects are related to:

- Missing input validation.
- Incomplete business rule enforcement.
- Negative scenario handling.
- Boundary value validation.
- Transaction verification logic.

9. Recommendations

The following improvements are recommended:

- ✓ Strong Password Validation
- ✓ Input Sanitization
- ✓ Boundary Value Validation
- ✓ Negative Amount Validation
- ✓ Enhanced Business Rule Validation
- ✓ Additional Negative Testing Coverage

10. Conclusion

A total of five defects and validation gaps were identified during testing activities.

One defect (PB-003) was resolved during automation enhancement activities, while the remaining defects represent opportunities for improving application robustness, security, and user experience.

The defect management process successfully utilized Jira for tracking and documentation, ensuring traceability and visibility throughout the testing lifecycle.